Introduction to Competitive Programming

Judges, verdicts, input/output, and your first accepted solutions

Learning goals

- Know what a programming contest is and how it is scored (ICPC vs. IOI vs. Codeforces-style)
- Create accounts on the main online judges and submit a solution
- Read a problem statement: story, input, output, constraints, samples
- Understand every judge verdict and what usually causes it
- Write correct C++ I/O for single tests, multiple tests, and read-until-EOF
- Avoid the classic first-week bugs: overflow, extra output, wrong format

What is competitive programming?

- Solve well-defined algorithmic problems **under time pressure**
- A program is judged automatically on **hidden tests**
- It must be **correct** on every test and **fast enough** (time limit) and **small enough** (memory limit)
- Skills: problem modeling, algorithms & data structures, careful implementation, debugging

> The goal of this course: go from "first AC" to solving ICPC regional problems by the end of the year.

Contest formats

Format	Who	Scoring
ICPC	teams of 3, 1 computer, 5 h	# solved, then penalty time (+20 min per wrong try)
IOI / olympiads	individual	partial points per subtask
Codeforces rounds	individual, ~2 h	points decay over time, hacks
AtCoder ABC/ARC/AGC	individual, ~100 min	points per problem, time tiebreak

Practice on all of them — they train different muscles.

The online judges we will use

- **Codeforces** (codeforces.com) — weekly rounds, huge archive, ratings
- **AtCoder** (atcoder.jp) — very clean statements, great Beginner Contests (ABC)
- **CSES Problem Set** (cses.fi/problemset) — 300+ classic problems, ordered by topic
- **Kattis** (open.kattis.com) — ICPC-style archive, many regional problems
- **USACO** (usaco.org) — training ladder Bronze → Platinum

Homework: make an account on each before next week.

Anatomy of a problem statement

```
Title, time limit (1 s), memory limit (256 MB)
Legend / story        <- what to compute, hidden in a story
Input                 <- exact format of stdin
Output                <- exact format of stdout
Constraints           <- 1 <= n <= 2*10^5, |a_i| <= 10^9  (READ THESE!)
Samples               <- small input/output pairs
Notes                 <- explanation of samples
```

The **constraints** tell you which algorithm is allowed (Week 3).

Example: CSES 1068 — Weird Algorithm

Given `n`, repeatedly: if `n` is even, `n /= 2`; else `n = 3n + 1`.
Print every value until `n == 1`. Constraint: `1 ≤ n ≤ 10^6`.

```
Input:  3
Output: 3 10 5 16 8 4 2 1
```

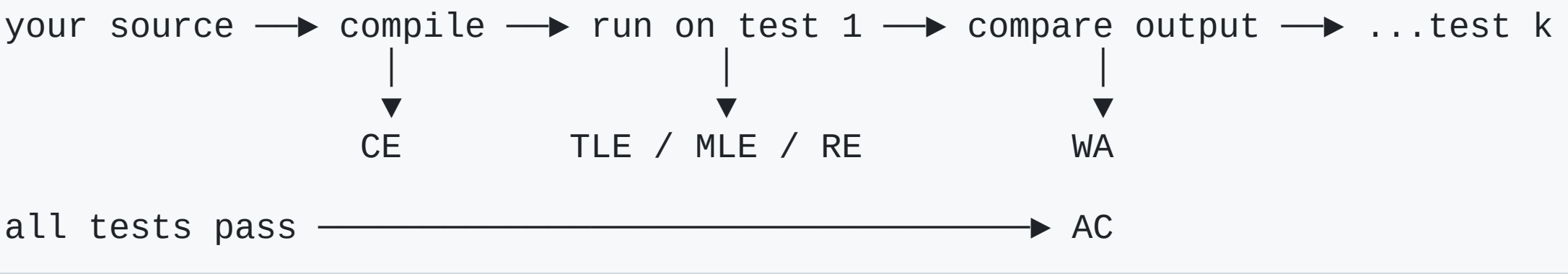
Looks trivial. What could go wrong?

Weird Algorithm — the trap

- Starting below 10^6, intermediate values exceed 2^31 − 1 ≈ 2.1·10^9
- `int` overflows → wrong answer or infinite loop
- Use `long long` (64-bit, up to ≈ 9.2·10^18)

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    long long n;
    cin >> n;
    cout << n;
    while (n != 1) {
        n = (n % 2 == 0) ? n / 2 : 3 * n + 1;
        cout << ' ' << n;
    }
    cout << '\n';
}
```

The judging pipeline



- Tests are run **in order**; the first failure is reported
- "Wrong answer on test 1" = you fail the sample — check locally first!

Verdicts (1/2)

Verdict	Meaning	Usual causes
AC	Accepted	—
WA	Wrong Answer	logic bug, overflow, wrong format, edge case (n=1)
TLE	Time Limit Exceeded	algorithm too slow, slow I/O, infinite loop
MLE	Memory Limit Exceeded	huge arrays, memory leak, deep recursion

Verdicts (2/2)

Verdict	Meaning	Usual causes
RE	Runtime Error	out-of-bounds index, division by 0, stack overflow, `return 1` from main
CE	Compilation Error	wrong language selected, missing header, typo
PE	Presentation Error (some judges)	extra/missing spaces or newlines
ILE / Idleness	Waiting for input	interactive problems: forgot to flush

Most judges ignore trailing whitespace — but never print **debug output** to stdout.

Rule of thumb: 10^8 simple operations per second

n	Max complexity that fits in ~1 s
≤ 10	O(n!)
≤ 20	O(2^n · n)
≤ 500	O(n^3)
≤ 5000	O(n^2)
≤ 2·10^5	O(n log n)
≤ 10^7	O(n)

We will justify this table in Week 3.

Input patterns: single test

```
int n;
cin >> n;
vector<long long> a(n);
for (auto &x : a) cin >> x;
```

`cin >>` skips **all** whitespace (spaces, newlines), so layout of the input does not matter for numbers.

Input patterns: multiple test cases

```
3           <- t
2           <- n of test 1
1 2
...
```

```
int t;
cin >> t;
while (t--) {
    int n;
    cin >> n;
    // solve one test, print its answer
}
```

Reset all global state between test cases!

Input patterns: read until end of file

Kattis / UVa often have no count of tests.

```
long long a, b;
while (cin >> a >> b) {            // stops at EOF or bad input
    cout << llabs(a - b) << '\n';
}
```

Whole lines (with spaces):

```
string line;
while (getline(cin, line)) { ... }
```

Mixing `cin >>` and `getline`? Call `cin.ignore()` / read the rest of the line first.

Fast I/O

```
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    ...
}
```

- Unties C++ streams from C stdio → `cin` becomes as fast as `scanf`
- Use `'\n'` instead of `endl` (`endl` flushes every line — slow)
- After this, **don't mix** `cin` with `scanf` / `printf`

Output and precision

```
cout << fixed << setprecision(6) << 3.14159265 << '\n'; // 3.141593
```

- Floating point answers are usually checked with tolerance (1e-6 relative/absolute)
- Print **exactly** what is asked: "YES"/"NO" vs "Yes"/"No" (check the statement!)
- Answers like "-1 if impossible" — test that path too

Integer overflow cheat sheet

Type	Range
`int`	±2.1·10^9
`long long`	±9.2·10^18
`unsigned long long`	0 … 1.8·10^19

- `n*(n+1)/2` with n = 2·10^5 → 2·10^10: needs `long long`
- `int a = 1e5; long long x = a * a;` **still overflows** (computed in `int`)
- Fix: `1LL * a * a`

Worked example: CSES 1083 — Missing Number

Numbers 1..n, one is missing, find it. n ≤ 2·10^5.

Idea: sum of 1..n is n(n+1)/2; subtract what we read.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    long long n; cin >> n;
    long long s = n * (n + 1) / 2;
    for (int i = 0; i < n - 1; i++) { long long x; cin >> x; s -= x; }
    cout << s << '\n';
}
```

O(n) time, O(1) memory.

Worked example: CF 4A — Watermelon

Can weight `w` be split into two **positive even** parts? (1 ≤ w ≤ 100)

- Tempting: "YES iff w even" → **WA on w = 2** (2 = 2 + 0, 0 not positive)
- Correct: `w % 2 == 0 && w > 2`

Lesson: **test the smallest inputs** by hand before submitting.

Workflow for every problem

1. Read the whole statement; underline constraints
2. Solve the samples **by hand**
3. Find an algorithm; check complexity against constraints
4. Code it; compile with warnings: `g++ -std=c++17 -O2 -Wall -o a a.cpp`
5. Run on samples: `./a < in1.txt`
6. Try edge cases: minimum n, maximum values, all equal
7. Submit; if not AC, re-read the statement before the code

Local setup

```
g++ -std=c++17 -O2 -Wall -Wextra -fsanitize=address,undefined -o a a.cpp
./a < sample.txt
diff <(./a < in.txt) out.txt && echo OK
```

- `-fsanitize` catches out-of-bounds and overflow while testing (slower, don't submit with it)
- Keep a **template file** (Week 2) to start every solution

Common first-week mistakes

- Using `int` where values exceed 2·10^9
- Printing prompts like `"Enter n:"` — the judge reads them as output → WA
- `endl` in a loop over 10^6 lines → TLE
- Forgetting to reset arrays/counters between test cases
- Wrong language/compiler selected → CE
- Reading the samples but not the constraints

Practice problems

- CSES 1068 Weird Algorithm · 1083 Missing Number · 1069 Repetitions · 1094 Increasing Array
- Codeforces 4A Watermelon · 71A Way Too Long Words · 231A Team · 158A Next Round · 282A Bit++
- AtCoder ABC086 A Product · ABC081 A Placing Marbles · ABC081 B Shift only
- Kattis `hello` · `carrots` · `r2`

Goal for this week: **10 accepted submissions** across at least 3 judges.

Summary

- Contests = hidden tests + time/memory limits + exact output format
- Verdicts tell you *what* failed; constraints tell you *what's allowed*
- Default to `long long`, fast I/O, `'\n'`
- Know the three input patterns: single, `t` tests, until EOF
- Next week: **C++ and the STL** — the toolbox for the rest of the year